

操作系统实验报告

Lab5:用户程序

信息安全

2313781 李胜林

2312796 张肇秋

2312323 杨中秀

一、实验目的

1. 了解第一个用户进程创建过程
2. 了解第一个用户进程创建过程
3. 了解ucore如何实现系统调用sys_fork/sys_exec/sys_exit/sys_wait来进行进程管理

二、实验内容

实验4完成了内核线程，但到目前为止，所有的运行都在内核态执行。实验5将创建用户进程，让用户进程在用户态执行，且在需要ucore支持时，可通过系统调用来让ucore提供服务。为此需要构造出第一个用户进程，并通过系统调用 `sys_fork / sys_exec / sys_exit / sys_wait` 来支持运行不同的应用程序，完成对用户进程的执行过程的基本管理。

三、练习

(一) 练习0：填写已有实验

本实验依赖实验2/3/4。请把你做的实验2/3/4的代码填入本实验中代码中有“LAB2”/“LAB3”/“LAB4”的注释相应部分。注意：为了能够正确执行 lab5 的测试应用程序，需对已完成的实验2/3/4的代码进行进一步改进。

(二) 练习1：加载应用程序并执行

`do_execv`函数调用 `load_icode`（位于 `kern/process/proc.c` 中）来加载并解析一个处于内存中的ELF执行文件格式的应用程序。需要补充 `load_icode` 的第6步，建立相应的用户内存空间来放置应用程序的代码段、数据段等，且要设置好 `proc_struct` 结构中的成员变量 `trapframe` 中的内容，确保在执行此进程后，能够从应用程序设定的起始执行地址开始执行。需设置正确的 `trapframe` 内容。

请在实验报告中简要说明你的设计实现过程。

请简要描述这个用户态进程被 `ucore` 选择占用 CPU 执行（`RUNNING` 态）到具体执行应用程序第一条指令的整个经过。

(三) 练习2：父进程复制自己的内存空间给子进程

创建子进程的函数 `do_fork` 在执行中将拷贝当前进程（即父进程）的用户内存地址空间中的合法内容到新进程中（子进程），完成内存资源的复制。具体是通过 `copy_range` 函数（位于 `kern/mm/pmm.c` 中）实现的，请补充 `copy_range` 的实现，确保能够正确执行。

请在实验报告中简要说明你的设计实现过程。

如何设计实现 `Copy on write` 机制？给出概要设计，鼓励给出详细设计。

(四) 练习3：阅读分析源代码，理解进程执行 fork/exec/wait/exit 的实现，以及系统调用的实现

请在实验报告中简要说明你对 fork/exec/wait/exit函数的分析。并回答如下问题：

1. 请分析fork/exec/wait/exit的执行流程。重点关注哪些操作是在用户态完成，哪些是在内核态完成？内核态与用户态程序是如何交错执行的？内核态执行结果是如何返回给用户程序的？
2. 请给出ucore中一个用户态进程的执行状态生命周期图（包执行状态，执行状态之间的变换关系，以及产生变换的事件或函数调用）。（字符方式画即可）

(五) 扩展练习 Challenge

1. 实现 Copy on Write (COW) 机制

给出实现源码,测试用例和设计报告（包括在cow情况下的各种状态转换（类似有限状态自动机）的说明）。

2. 说明该用户程序是何时被预先加载到内存中的？与我们常用操作系统的加载有何区别，原因是什么？

四、练习解答

(一) 练习0：填写已有实验

对于lab2，实验代码中已经完成了填写，我们可以直接略过。对于lab3，这次需要填写的代码与上次实验一样，我们不需要做出修改，只需要复制粘贴进去即可。对于lab4的代码，对于 `alloc_proc` 和 `do_fork` 函数我们需要进行一定的修改，具体修改如下所示：

`alloc_proc`

```
1 // LAB4:EXERCISE1 2312323
2 // 初始化进程状态为未初始化
3     proc->state = PROC_UNINIT;
4     // 初始化进程ID为-1（无效ID）
5     proc->pid = -1;
6     // 使用内核页目录表的基址
7     proc->pgdir = 0;//turned into uninit status
8     // 初始化运行次数为0
9     proc->runs = 0;
10    // 初始化内核栈地址为0
11    proc->kstack = 0;
12    // 初始化不需要重新调度
13    proc->need_resched = 0;
14    // 初始化父进程指针为NULL
15    proc->parent = NULL;
16    // 初始化内存管理结构为NULL
17    proc->mm = NULL;
18    // 初始化上下文结构（全部设为0）
19    memset(&(proc->context), 0, sizeof(struct context));
20    // 初始化陷阱帧指针为NULL
21    proc->tf = NULL;
22    // 初始化进程标志为0
23    proc->flags = 0;
24    // 初始化进程名称为空字符串
```

```

25     memset(proc->name, 0, PROC_NAME_LEN + 1);
26     // 初始化等待状态为0
27     proc->wait_state = 0;
28     // 初始化进程关系指针为NULL
29     proc->cptr = NULL;
30     proc->optr = NULL;
31     proc->yptr = NULL;

```

在这里，我们添加了下面这四行，分别是对应的将等待状态初始化为0.然后将进程的三个关系指针初始化NULL。

do_fork

```

1 // LAB4:EXERCISE2 2313781
2 // 1. call exit_mmap & put_pgdir & mm_destroy to free the almost all
   memory space of process
3     if ((proc = alloc_proc()) == NULL) {
4         goto fork_out;
5     }
6     proc->parent=current;
7     assert(current->wait_state==0);
8
9     // 2. call setup_kstack to allocate a kernel stack for child process
10    if (setup_kstack(proc) != 0) {
11        goto bad_fork_cleanup_proc;
12    }
13    // 3. call copy_mm to dup OR share mm according clone_flag
14    if (copy_mm(clone_flags, proc) != 0) {
15        goto bad_fork_cleanup_kstack;
16    }
17    // 4. call copy_thread to setup tf & context in proc_struct
18    //here we set the context.re<-forkret
19    copy_thread(proc, stack, tf);
20    // 5. insert proc_struct into hash_list && proc_list, set the
   relation links of process
21    bool intr_flag;
22    local_intr_save(intr_flag); // 关中断，保护临界区
23    proc->pid = get_pid();
24    hash_proc(proc);
25    set_links(proc); //add in lab5
26    local_intr_restore(intr_flag);
27    // 6. call wakeup_proc to make the new child process RUNNABLE
28    wakeup_proc(proc);
29    // 7. set ret vaule using child proc's pid
30    ret = proc->pid;

```

这里我们主要是原来的将进程链入进程链表的和更新当前进程的数量这两个步骤删除了。然后我们新增了 `set_links(proc)`；然后调整了下面两行代码的位置：

```

1 proc->parent=current;
2 assert(current->wait_state==0);

```

用以匹配在注释中给的提示：

- 1 pdate step 1: set child proc's parent to current process, make sure current process's wait_state is 0
- 2 update step 5: insert proc_struct into hash_list && proc_list, set the relation links of process

这样我们的代码就补全了，下一步便可以进行后面的练习了。

(二) 练习1：加载应用程序并执行

在这里我们主要的任务是完成第 (6) 步中的代码，这一步主要是实现为用户进程设置中断帧。目前，框架中已经完成了中断帧的声明和设置 `sstatus` 的步骤，接下来我们需要进行下面几个步骤：

1. **设置用户栈指针 (sp)**：将用户进程的栈顶指针设置为用户栈的顶部地址
2. **设置程序计数器(epc)**：将用户进程的入口点设置为ELF文件的入口地址
3. **设置特权级别**:清除SPP位，确保 `sret` 指令返回到用户模式(U-mode)
4. **启用中断**:设置SPIE位，确保返回到用户模式后中断被启用

```
1  /* LAB5:EXERCISE1 2312323*/
2      // user stack top
3      tf->gpr.sp = USTACKTOP;
4      // entry point of user program (ELF header entry)
5      // elf 在 load_icode 一开始就定义了: struct elfhdr *elf = (struct elfhdr
6      *)binary;
7      tf->epc = elf->e_entry;
8      // setup sstatus for returning to user mode
9      // 1. 清掉 SPP 位 (表示 sret 返回到 U 模式而不是 S 模式)
10     // 2. 置位 SPIE 位 (在 sret 后开启用户态中断)
11     sstatus &= ~SSTATUS_SPP;    // 清 SPP
12     sstatus |= SSTATUS_SPIE;    // 置 SPIE
13     tf->status = sstatus;
14     ret = 0;
```

请简要描述这个用户态进程被 `ucore` 选择占用 CPU 执行 (RUNNING 态) 到具体执行应用程序第一条指令的整个经过。

(1)首先是时钟中断等触发调度

发生时钟中断后进入 `trap()`，然后会根据当前进程 `current` 是否存在分别进行处理，如果 `current` 为空，则不需要保存/恢复 `current->tf` 和做用户态 `resched` 检查，否则则获取当前的进程的中断帧，根据这个中断帧是否是S态终端进行分类处理，如果不是内核中断（即为用户态触发的中断），说明这是个用户进程，我们就继续查看这个进程的 `need_resched`，如果 `need_resched` 为 `true`，说明该用户进程需要进行调度，我们就进入 `schedule()` 函数中进行处理。

```
1  void trap(struct trapframe *tf)
2  {
3      // dispatch based on what type of trap occurred
4      //  cputs("some trap");
5      if (current == NULL)
6      {
7          trap_dispatch(tf);
8      }
9      else
```

```

10     {
11         struct trapframe *otf = current->tf;
12         current->tf = tf;
13         bool in_kernel = trap_in_kernel(tf);
14         trap_dispatch(tf);
15         current->tf = otf;
16         if (!in_kernel)
17         {
18             if (current->flags & PF_EXITING)
19             {
20                 do_exit(-E_KILLED);
21             }
22             if (current->need_resched)
23             {
24                 schedule();
25             }
26         }
27     }
28 }

```

(2)然后是调度器让 next 变为 RUNNING 并切换

```

1 void schedule(void)
2 {
3     bool intr_flag;
4     list_entry_t *le, *last;
5     struct proc_struct *next = NULL;
6     local_intr_save(intr_flag);
7     {
8         current->need_resched = 0;
9         last = (current == idleproc) ? &proc_list : &(current->list_link);
10        le = last;
11        do
12        {
13            if ((le = list_next(le)) != &proc_list)
14            {
15                next = le2proc(le, list_link);
16                if (next->state == PROC_RUNNABLE)
17                {
18                    break;
19                }
20            }
21        } while (le != last);
22        if (next == NULL || next->state != PROC_RUNNABLE)
23        {
24            next = idleproc;
25        }
26        next->runs++;
27        if (next != current)
28        {
29            proc_run(next);
30        }
31    }
32    local_intr_restore(intr_flag);
33 }

```

这里的函数与我们lab4中的逻辑相似，便不再赘述了，通过这个函数，可以找到下一个可执行的进程，即我们需要调度的用户进程。将其从 RUNNABLE 语义上转为 RUNNING，然后调用 `proc_run(next)`。这个函数我们在lab4中已经完成补充了，我们之后便会进入这个函数中。

```
1 void proc_run(struct proc_struct *proc)
2 {
3     if (proc != current)
4     {
5         // LAB4:EXERCISE3 2312796
6         bool intr_flag;
7         struct proc_struct *prev = current; // save current proc//Line 77
8         struct proc_struct *next = proc;    // set proc to switch
9         // ban trap
10        local_intr_save(intr_flag);
11        // switch nowa proc to which to run
12        current = proc;
13        // switch pgdir,to new space
14        // as for kernel t,pgdir could be NULL,employ boot_pgdir_pa
15        if (proc->pgdir != 0) {
16            lsatp(proc->pgdir); // load padir 2 satp of new proc
17        } else {
18            lsatp(boot_pgdir_pa); // pgdir of kernel t
19        }
20        // call switch
21        switch_to(&(prev->context), &(next->context));
22        local_intr_restore(intr_flag);
23    }
24 }
```

(3)第三步是内核上下文切换完成

在 `proc_run` 函数中，我们调用了 `switch_to` 函数，在这里保存了原进程和新进程的14个重要寄存器的状态，包括12个通用寄存器，一个 `sp`，一个 `ra`。而 `switch_to` 最后的 `ret` 会跳到 `next->context.ra` 指向的位置，因此第一次运行的新进程会落到 `forkrets`。

```
1 .globl switch_to
2 switch_to:
3     # save from's registers
4     STORE ra, 0*REGBYTES(a0)
5     STORE sp, 1*REGBYTES(a0)
6     STORE s0, 2*REGBYTES(a0)
7     STORE s1, 3*REGBYTES(a0)
8     STORE s2, 4*REGBYTES(a0)
9     STORE s3, 5*REGBYTES(a0)
10    STORE s4, 6*REGBYTES(a0)
11    STORE s5, 7*REGBYTES(a0)
12    STORE s6, 8*REGBYTES(a0)
13    STORE s7, 9*REGBYTES(a0)
14    STORE s8, 10*REGBYTES(a0)
15    STORE s9, 11*REGBYTES(a0)
16    STORE s10, 12*REGBYTES(a0)
17    STORE s11, 13*REGBYTES(a0)
18
19    # restore to's registers
```

```

20     LOAD ra, 0*REGBYTES(a1)
21     LOAD sp, 1*REGBYTES(a1)
22     LOAD s0, 2*REGBYTES(a1)
23     LOAD s1, 3*REGBYTES(a1)
24     LOAD s2, 4*REGBYTES(a1)
25     LOAD s3, 5*REGBYTES(a1)
26     LOAD s4, 6*REGBYTES(a1)
27     LOAD s5, 7*REGBYTES(a1)
28     LOAD s6, 8*REGBYTES(a1)
29     LOAD s7, 9*REGBYTES(a1)
30     LOAD s8, 10*REGBYTES(a1)
31     LOAD s9, 11*REGBYTES(a1)
32     LOAD s10, 12*REGBYTES(a1)
33     LOAD s11, 13*REGBYTES(a1)
34
35     ret

```

在这一步后，CPU 已经在next 的内核栈和内核执行流中继续运行了。

(4)第四步是准备 next 进程的用户态返回现场 trapframe

当进程被 do_execve 函数装载用户程序时，load_icode 函数（太长了不粘贴进来了）做了两件关键事：

一是建好并切到该进程用户页表

```

1  current->pgdir = PADDR(mm->pgdir);
2  lsatp(PADDR(mm->pgdir));

```

二是我们在上面补充的第六步的代码，通过这样进程的中断帧就已经是“回到用户态就从入口开始跑”的样子。

(5)第五步是走 forkrets / __trapret / sret 部分返回

由于 next->context.ra 在创建时就被设成 forkrets（第一次运行），next->context.sp 也指向 trapframe；因此 ret 会直接跳到 forkrets。

```

1  __trapret:
2      RESTORE_ALL
3      # return from supervisor call
4      sret
5
6      .globl forkrets
7  forkrets:
8      # set stack to this new process's trapframe
9      move sp, a0
10     j __trapret

```

最后通过 sret 使得 CPU 从 S 特权级返回值 U 特权级，此时的 PC 为 trapframe 的 epc（即用户程序入口）；SP 为 trapframe 的 gpr.sp（即用户栈顶）。这样从下一步开始，CPU 就在用户页表下取指并执行应用程序的第一条指令了。

(三) 练习2：父进程复制自己的内存空间给子进程

在这里我们主要实现的是页面的复制以及映射的建立过程，前面的代码已经帮我们实现了大部分内容了，所以我们现在需要的就是下面三步：

获取内核虚拟地址

```
1  uintptr_t *src_kvaddr = page2kva(page);      // 源页面的内核虚拟地址
2  uintptr_t *dst_kvaddr = page2kva(npage);     // 目标页面的内核虚拟地址
```

在这里我们通过 `page2kva()` 将物理页面转换为内核虚拟地址，在内核空间中，所有物理页面都有直接映射的虚拟地址，这样内核就可以像访问普通内存一样访问物理页面。这里需要注意的是虚拟地址的类型为 `uintptr_t` 而不是空洞的 `void`，如果使用 `void` 在进行测试时，结果会产生些许问题。

内存复制

```
1  memcpy(dst_kvaddr, src_kvaddr, PGSIZE);
```

这里我们便直接调用 `memcpy` 函数进行内存复制即可了。

建立映射

```
1  ret = page_insert(to, npage, start, perm);
```

我们使用 `page_insert` 建立物理页面到虚拟地址的映射，其中 `to` 为目标页表，`npage` 为新的物理页面，`start` 为进程的虚拟地址，`perm` 为页面权限。下面是整体代码的实现：

```
1  int copy_range(pde_t *to, pde_t *from, uintptr_t start, uintptr_t end,
2                bool share)
3  {
4      assert(start % PGSIZE == 0 && end % PGSIZE == 0);
5      assert(USER_ACCESS(start, end));
6      // copy content by page unit.
7      do
8      {
9          // call get_pte to find process A's pte according to the addr start
10         pte_t *ptep = get_pte(from, start, 0), *nptep;
11         if (ptep == NULL)
12         {
13             start = ROUNDDOWN(start + PTSIZE, PTSIZE);
14             continue;
15         }
16         // call get_pte to find process B's pte according to the addr start.
17         If
18         // pte is NULL, just alloc a PT
19         if (*ptep & PTE_V)
20         {
21             if ((nptep = get_pte(to, start, 1)) == NULL)
22             {
23                 return -E_NO_MEM;
24             }
25             uint32_t perm = (*ptep & PTE_USER);
26             // get page from ptep
```



```

26         struct Page *page = pte2page(*ptep);
27         // alloc a page for process B
28         struct Page *npage = alloc_page();
29         assert(page != NULL);
30         assert(npage != NULL);
31         int ret = 0;
32         /* LAB5:EXERCISE2 2312796
33          */
34         // (1) kernel virtual address of src page
35         uintptr_t *src_kvaddr = page2kva(page);
36         // (2) kernel virtual address of new page
37         uintptr_t *dst_kvaddr = page2kva(npage);
38
39         // (3) copy one page content
40         memcpy(dst_kvaddr, src_kvaddr, PGSIZE);
41
42         // (4) insert mapping into dst pgdir
43         ret = page_insert(to, npage, start, perm);
44         assert(ret == 0);
45     }
46     start += PGSIZE;
47 } while (start != 0 && start < end);
48 return 0;
49 }

```

问题：如何设计实现 Copy on Write 机制？给出概要设计

在创建子进程时不立即复制父进程的内存页，而是让父子进程先共享同一批物理页。为了保证共享期间不发生相互覆盖，内核把这些原本可写的页在父子两边都暂时改成只读，并在页表里做一个“写时复制”的标记，同时维护物理页被多少进程共享的计数。这样父子进程可以正常读取共享页；当任意一方第一次尝试写入某个共享页时，处理器会因为只读保护触发写入异常，内核在异常处理中检查到该页带有写时复制标记后，根据共享计数决定动作：如果该页仍被多个进程共享，就分配一个新的物理页，把原内容复制过去，并让当前进程的映射指向新页且恢复为可写；如果该页已经只被当前进程独占，就无需复制，直接把它恢复为可写即可。进程退出或解除映射时相应减少共享计数，计数归零后再回收物理页。通过这种“先共享、后按需复制”的策略，既保证正确性，又显著降低 fork 的时间和内存开销。

（四）练习3：阅读分析源代码，理解进程执行 fork/exec/wait/exit 的实现，以及系统调用的实现

1.请分析fork/exec/wait/exit的执行流程。重点关注哪些操作是在用户态完成，哪些是在内核态完成？内核态与用户态程序是如何交错执行的？内核态执行结果是如何返回给用户程序的？

fork

sys_fork函数

```

1 static int
2 sys_fork(uint64_t arg[]) {
3     struct trapframe *tf = current->tf;
4     uintptr_t stack = tf->gpr.sp;
5     return do_fork(0, stack, tf);
6 }

```

`sys_fork` 是一个系统调用处理函数，它的核心功能是创建一个与当前进程完全相同的新子进程。该函数通过获取当前进程的陷阱帧 `tf`，从中提取出用户栈指针 `sp`，然后将这些信息连同标志位一起传递给 `do_fork` 函数来执行实际的进程复制工作。

do_fork函数

```
1  int do_fork(uint32_t clone_flags, uintptr_t stack, struct trapframe *tf)
2  {
3      int ret = -E_NO_FREE_PROC;
4      struct proc_struct *proc;
5      if (nr_process >= MAX_PROCESS)
6      {
7          goto fork_out;
8      }
9      ret = -E_NO_MEM;
10     if ((proc = alloc_proc()) == NULL) {
11         goto fork_out;
12     }
13     proc->parent=current;
14     assert(current->wait_state==0);
15     if (setup_kstack(proc) != 0) {
16         goto bad_fork_cleanup_proc;
17     }
18     if (copy_mm(clone_flags, proc) != 0) {
19         goto bad_fork_cleanup_kstack;
20     }
21     copy_thread(proc, stack, tf);
22     bool intr_flag;
23     local_intr_save(intr_flag);
24     proc->pid = get_pid();
25     hash_proc(proc);
26     set_links(proc);
27     local_intr_restore(intr_flag);
28     wakeup_proc(proc);
29     ret = proc->pid;
30 fork_out:
31     return ret;
32 bad_fork_cleanup_kstack:
33     put_kstack(proc);
34 bad_fork_cleanup_proc:
35     kfree(proc);
36     goto fork_out;
37 }
```

`do_fork` 函数是操作系统中创建新进程的核心实现，它负责完整地复制当前进程来生成一个子进程。该函数首先检查系统进程数量限制，然后为子进程分配进程控制块并设置其父进程为当前进程。接着，它通过 `setup_kstack` 为子进程分配内核栈，通过 `copy_mm` 复制或共享内存空间，并通过 `copy_thread` 设置陷阱帧和上下文来确保子进程能够正确开始执行。在完成这些资源分配后，函数会为子进程分配PID并将其加入进程哈希表和管理链表，最后唤醒子进程使其进入就绪状态。

exec

sys_exec函数

```
1 static int
2 sys_exec(uint64_t arg[]) {
3     const char *name = (const char *)arg[0];
4     size_t len = (size_t)arg[1];
5     unsigned char *binary = (unsigned char *)arg[2];
6     size_t size = (size_t)arg[3];
7     return do_execve(name, len, binary, size);
8 }
```

`sys_exec` 函数是执行新程序的系统调用接口，它负责将当前进程完全替换为一个全新的程序映像。该函数从用户空间传递的参数中提取程序名称、长度、二进制代码指针和大小等信息，然后调用 `do_execve` 函数来执行实际的程序加载和替换工作。

do_exec函数

```
1 int do_execve(const char *name, size_t len, unsigned char *binary, size_t
   size)
2 {
3     struct mm_struct *mm = current->mm;
4     if (!user_mem_check(mm, (uintptr_t)name, len, 0))
5     {
6         return -E_INVAL;
7     }
8     if (len > PROC_NAME_LEN)
9     {
10         len = PROC_NAME_LEN;
11     }
12     char local_name[PROC_NAME_LEN + 1];
13     memset(local_name, 0, sizeof(local_name));
14     memcpy(local_name, name, len);
15     if (mm != NULL)
16     {
17         cputs("mm != NULL");
18         lsatp/boot_pgdir_pa);
19         if (mm_count_dec(mm) == 0)
20         {
21             exit_mmap(mm);
22             put_pgdir(mm);
23             mm_destroy(mm);
24         }
25         current->mm = NULL;
26     }
27     int ret;
28     if ((ret = load_icode(binary, size)) != 0)
29     {
30         goto execve_exit;
31     }
32     set_proc_name(current, local_name);
33     return 0;
34 execve_exit:
35     do_exit(ret);
```

```

36     panic("already exit: %e.\n", ret);
37 }
38

```

`do_execve` 函数是执行新程序的核心实现，它负责将当前进程的内存空间完全替换为新的程序映像。该函数首先进行安全性检查，验证用户传入的程序名称和参数的合法性，并复制程序名称到内核空间。接着，它会清理进程现有的内存管理结构，如果存在原有内存映射则通过 `exit_mmap` 和 `mm_destroy` 等函数释放所有相关资源。然后调用 `load_icode` 函数来实际加载和解析可执行文件二进制代码，建立新的代码段、数据段和堆栈映射。如果程序加载成功，函数会设置新的进程名称并返回，让进程开始执行新程序；如果加载失败，则通过 `do_exit` 终止进程并返回错误码。

wait

sys_wait函数

```

1  static int
2  sys_wait(uint64_t arg[]) {
3      int pid = (int)arg[0];
4      int *store = (int *)arg[1];
5      return do_wait(pid, store);
6  }

```

`sys_wait` 函数是进程等待机制的系统调用接口，它允许父进程等待其子进程的终止并回收系统资源。该函数接收两个参数：要等待的特定子进程的PID（如果为-1则表示等待任意子进程），以及一个用于存储子进程退出状态码的指针。它通过调用 `do_wait` 函数来实际执行等待操作。

do_wait函数

```

1  int do_wait(int pid, int *code_store)
2  {
3      struct mm_struct *mm = current->mm;
4      if (code_store != NULL)
5      {
6          if (!user_mem_check(mm, (uintptr_t)code_store, sizeof(int), 1))
7          {
8              return -E_INVALID;
9          }
10     }
11     struct proc_struct *proc;
12     bool intr_flag, haskid;
13 repeat:
14     haskid = 0;
15     if (pid != 0)
16     {
17         proc = find_proc(pid);
18         if (proc != NULL && proc->parent == current)
19         {
20             haskid = 1;
21             if (proc->state == PROC_ZOMBIE)
22             {
23                 goto found;
24             }
25         }
26     }

```

```

27     else
28     {
29         proc = current->cptr;
30         for (; proc != NULL; proc = proc->optr)
31         {
32             haskid = 1;
33             if (proc->state == PROC_ZOMBIE)
34             {
35                 goto found;
36             }
37         }
38     }
39     if (haskid)
40     {
41         current->state = PROC_SLEEPING;
42         current->wait_state = WT_CHILD;
43         schedule();
44         if (current->flags & PF_EXITING)
45         {
46             do_exit(-E_KILLED);
47         }
48         goto repeat;
49     }
50     return -E_BAD_PROC;
51 found:
52     if (proc == idleproc || proc == initproc)
53     {
54         panic("wait idleproc or initproc.\n");
55     }
56     if (code_store != NULL)
57     {
58         *code_store = proc->exit_code;
59     }
60     local_intr_save(intr_flag);
61     {
62         unhash_proc(proc);
63         remove_links(proc);
64     }
65     local_intr_restore(intr_flag);
66     put_kstack(proc);
67     kfree(proc);
68     return 0;
69 }

```

`do_wait` 函数负责父进程等待和回收子进程资源。它首先检查子进程状态：如果指定PID或任意子进程已处于僵尸状态，则立即回收其内核栈和进程控制块，并返回退出码；如果子进程仍在运行，则让父进程进入睡眠等待状态，稍后被唤醒时重新检查。这确保了父进程能正确同步子进程结束，防止僵尸进程产生。

exit

sys_exit函数

```
1 static int
2 sys_exit(uint64_t arg[]) {
3     int error_code = (int)arg[0];
4     return do_exit(error_code);
5 }
```

`sys_exit` 函数是进程终止的系统调用接口，它接收一个错误代码作为参数，并调用 `do_exit` 函数来执行实际的进程退出流程。

do_exit函数

```
1 int do_exit(int error_code)
2 {
3     if (current == idleproc)
4     {
5         panic("idleproc exit.\n");
6     }
7     if (current == initproc)
8     {
9         panic("initproc exit.\n");
10    }
11    struct mm_struct *mm = current->mm;
12    if (mm != NULL)
13    {
14        lsatp(boot_pgdir_pa);
15        if (mm_count_dec(mm) == 0)
16        {
17            exit_mmap(mm);
18            put_pgdir(mm);
19            mm_destroy(mm);
20        }
21        current->mm = NULL;
22    }
23    current->state = PROC_ZOMBIE;
24    current->exit_code = error_code;
25    bool intr_flag;
26    struct proc_struct *proc;
27    local_intr_save(intr_flag);
28    {
29        proc = current->parent;
30        if (proc->wait_state == WT_CHILD)
31        {
32            wakeup_proc(proc);
33        }
34        while (current->cptr != NULL)
35        {
36            proc = current->cptr;
37            current->cptr = proc->optr;
38            proc->yptr = NULL;
39            if ((proc->optr = initproc->cptr) != NULL)
40            {
```

```

41         initproc->cptr->yptr = proc;
42     }
43     proc->parent = initproc;
44     initproc->cptr = proc;
45     if (proc->state == PROC_ZOMBIE)
46     {
47         if (initproc->wait_state == WT_CHILD)
48         {
49             wakeup_proc(initproc);
50         }
51     }
52 }
53 }
54 local_intr_restore(intr_flag);
55 schedule();
56 panic("do_exit will not return!! %d.\n", current->pid);
57 }
58

```

`do_exit` 函数处理进程退出。它释放进程内存资源，将状态设为僵尸进程，保存退出码并唤醒等待的父进程。同时将子进程过继给初始化进程，最后调度其他进程运行，完成进程生命周期管理。

问题回答

(1) 在执行流程中，子进程调用 `fork`、`exec`、`wait`、`exit` 以及到 `ecall` 都是在用户态进行的。而从内核 `trap` 入口到分发系统调用 (`trap_dispatch`) 再到上面分析的这些 `sys`、`do` 函数，最后到将返回值写入中断帧 (`trapframe`) 都是在内核状态进行的。最后在调用 `sret` 后，又重新返回用户态。

(2) 内核态到用户态是通过 `sret` 实现的，而用户态到内核态是通过 `ecall` 实现的。

(3) 内核在处理完系统调用后，把返回值写进 `trapframe` 对应位置，随后调用 `sret` 返回用户态，用户态通过 `trapframe` 获取对应的返回值。

2.请给出ucore中一个用户态进程的执行状态生命周期图（包括执行状态，执行状态之间的变换关系，以及产生变换的事件或函数调用）。（字符方式画即可）

```

1      [ start / create request ]
2      (proc_init 创建 initproc / idleproc
3      或父进程 do_fork 创建子进程)
4          |
5          v
6
7      [ PROC_UNINIT ]  uninitialized  --  alloc_proc()
8          |
9          | do_fork():
10         |   |-- alloc_proc()           分配 PCB
11         |   |-- setup_kstack()        分配内核栈
12         |   |-- copy_mm()/dup_mmap()  复制/共享地址空间
13         |   |-- copy_thread()         准备 trapframe/context
14         |   |-- wakeup_proc()         置 RUNNABLE + 入队
15         |   v
16
17      [ PROC_RUNNABLE ]  runnable/ready -- wakeup_proc(), proc_init()
18          |
19          | schedule():

```

```

20 |   └─ proc_run(next):
21 |       └─ current = next
22 |       └─ lsatp(next->pgdir)    切换页表
23 |       └─ switch_to(prev,next)  内核上下文切换
24 |   v
25 |
26 | ┌ PROC_RUNNING ─ running on CPU -- proc_run()/switch_to()
27 | └─
28 |   |
29 |   | (A) 时间片到/需重调度/主动让出
30 |   |   timer interrupt → trap()
31 |   |   → trap_dispatch()
32 |   |   → current->need_resched = 1
33 |   |   trap() 返回前检查 need_resched
34 |   |   → schedule()
35 |   v
36 |   ┌ PROC_RUNNABLE ─
37 |   └─
38 |   |
39 |   | (B) 等待事件 → 阻塞
40 |   |   sleep_on()/do_wait()/do_sleep()/同步原语 wait:
41 |   |   └─ current->state = PROC_SLEEPING
42 |   |   └─ current->wait_state = ...
43 |   |   └─ schedule()
44 |   v
45 |
46 | ┌ PROC_SLEEPING ─ sleeping/blocked -- do_wait(), do_sleep(),
47 | │ try_free_pages()
48 | └─
49 |   | 事件发生 → 唤醒
50 |   |  wakeup_proc(p):
51 |   |   └─ p->state = PROC_RUNNABLE
52 |   |   └─ 入就绪队列
53 |   |   例: 子进程 do_exit() 唤醒父进程
54 |   |   I/O 中断完成 / semaphore_up() / cond_signal()
55 |   v
56 |   ┌ PROC_RUNNABLE ─
57 |   └─
58 |   | (再次被 schedule/proc_run 选中)
59 |   v
60 |
61 | ┌ PROC_RUNNING ─
62 | └─
63 |   |
64 |   | exit 系统调用/异常退出
65 |   | sys_exit() → do_exit(code):
66 |   |   └─ exit_mmap()/mm_destroy() 释放用户空间
67 |   |   └─ 释放文件等资源
68 |   |   └─ current->state = PROC_ZOMBIE
69 |   |   └─ wakeup_proc(parent) 若父在 wait
70 |   |   └─ schedule()          切走
71 |   v
72 |   ┌ PROC_ZOMBIE ─ almost dead -- do_exit()
73 |   └─
74 |

```



```

75 |         | 父进程 wait 回收
76 |         | sys_wait() → do_wait():
77 |         |   └─ 找到 zombie 子进程
78 |         |   └─ 取 exit_code
79 |         |   └─ unhash_proc()/remove_links()
80 |         |   └─ proc_free(child) 释放 PCB/内核栈
81 |         v
82 |
83 | ┌ FREED ────────────┐ dead/freed -- proc_free()
84 | └───────────────────┘
85 |

```

(五) 扩展Challenge

1.实现 Copy on Write (COW) 机制

设计思路

1. fork 时不再复制物理页，父子进程共享同一物理页；
2. 对原本可写的用户页，父子双方都清掉 PTE_W 并打上 PTE_COW 标记；
3. 当子/父第一次写该页时触发Store page fault，在异常里检测 PTE_COW:
 - 若 ref>1: 分配新页、拷贝、改成可写、清掉 COW
 - 若 ref==1: 直接改回可写、清掉 COW

代码实现

riscv.h: 在该文件中增加软件位

在 RISC-V 页表里 PTE_SOFT 0x300 有两个软件保留位，我们用其中一个做 COW 标志。

```

1 | #define PTE_SOFT 0x300 // Reserved for Software
2 | #define PTE_COW  0x100 // Copy-On-write (software)

```

vmm.c: 在复制时，实现“共享+COW”路径

在这个函数中，我们仅修改一行，如下所示：

```

1 | bool share = 1; // enable COW for forked child

```

由于 copy_mm 里非 CLONE_VM 的 fork 会调用 dup_mmap，因此我们在这里改完就能让 fork 进入 COW。

pmm.c: 在 copy_range 里实现 COW 共享映射

我们在原来的EXERCISE2的位置实现，原来传进来的参数中就存在这share，在这里我们把share应用起来，改成下面这样的代码：

```

1 | /* LAB5:EXERCISE2 2312796
2 |  * replicate content of page to npage, build the map of phy addr of
3 |  * nage with the linear addr start
4 |  *
5 |  * Some Useful MACROs and DEFINES, you can use them in below
6 |  * implementation.

```

```

7  * MACROS or Functions:
8  *   page2kva(struct Page *page): return the kernel virtual addr of
9  *   memory which page managed (SEE pmm.h)
10 *   page_insert: build the map of phy addr of an Page with the
11 *   linear addr la
12 *   memcpy: typical memory copy function
13 *
14 * (1) find src_kvaddr: the kernel virtual address of page
15 * (2) find dst_kvaddr: the kernel virtual address of npage
16 * (3) memory copy from src_kvaddr to dst_kvaddr, size is PGSIZE
17 * (4) build the map of phy addr of npage with the linear addr start
18 */
19 if (share) {
20 // Only writable pages need COW. Read-only pages can be shared directly.
21 if (perm & PTE_W) {
22     perm = (perm & ~PTE_W) | PTE_COW;
23     // update parent's mapping to be COW as well
24     *ptep = (*ptep & ~PTE_W) | PTE_COW;
25     tlb_invalidate(from, start);
26 }
27 ret = page_insert(to, page, start, perm);
28 } else {
29 // deep copy
30 void *src_kvaddr = page2kva(page);
31 void *dst_kvaddr = page2kva(npage);
32 memcpy(dst_kvaddr, src_kvaddr, PGSIZE);
33 ret = page_insert(to, npage, start, perm);
34 }

```

这样，我们就能根据share的值选择原来默认的方式或者是现在的COW方式了。

trap.c: 在 Store Page Fault 里做“写时复制”

在这里我们找到 `case CAUSE_STORE_PAGE_FAULT:` 分支，然后将其代码改成这个形式即可：

```

1  case CAUSE_STORE_PAGE_FAULT:{
2      uintptr_t badaddr = read_csr(stval);
3      uintptr_t la = ROUNDDOWN(badaddr, PGSIZE);
4
5      if (current != NULL && current->mm != NULL) {
6          pte_t *ptep = get_pte(current->mm->pgdir, badaddr, 0);
7          if (ptep != NULL && (*ptep & PTE_V) && (*ptep & PTE_COW)) {
8              struct Page *page = pte2page(*ptep);
9
10             if (page_ref(page) > 1) {
11                 struct Page *npage = alloc_page();
12                 if (npage == NULL) panic("COW: out of memory");
13                 memcpy(page2kva(npage), page2kva(page), PGSIZE);
14
15                 uint32_t perm = (*ptep & PTE_USER);
16                 perm = (perm | PTE_W) & ~PTE_COW;
17                 page_insert(current->mm->pgdir, npage, la, perm);
18             } else {
19                 *ptep = (*ptep | PTE_W) & ~PTE_COW;
20                 tlb_invalidate(current->mm->pgdir, la);

```

```

21     }
22     break; // handled
23 }
24 }
25 cprintf("Store/AMO page fault\n");
26 break;
27 }

```

这里是实现COW的核心部分，我们的实现逻辑是：

首先使用 `read_csr(stval)` 取 fault 地址（RISC-V 的 `badaddr`），然后判断 `PTE_COW` 才是 COW fault；否则仍按原逻辑报错。接着根据 `ref` 的大小来进行判断，若 `ref>1`：复制新页后 `page_insert` 会替换映射，并把旧页 `ref--`；若 `ref==1`：不用复制，直接把页改回可写即可，最后 `tlb_invalidate` 保证新权限生效。

测试用例

在测试样例中，我使用了四个文件进行测试

cow_basic.c

```

1  #include <ulib.h>
2  #include <stdio.h>
3  #include <unistd.h>
4
5
6  static volatile int x = 1;
7
8  int main(void) {
9      cprintf("[cow_basic] start pid=%d x=%d (addr=%p)\n", getpid(), x, &x);
10
11     int pid = fork();
12     if (pid == 0) {
13         // child
14         cprintf("[cow_basic-child] pid=%d before write x=%d\n", getpid(),
15 x);
16         x = 2; // should trigger COW
17         cprintf("[cow_basic-child] pid=%d after write x=%d\n", getpid(),
18 x);
19         exit(0);
20     } else {
21         // parent
22         waitpid(pid, NULL);
23         cprintf("[cow_basic-parent] pid=%d after child exit x=%d\n",
24 getpid(), x);
25     }
26     return 0;
27 }
28
29 /*
30 T1: fork 后共享页、写触发复制
31 fork 之后，child 第一次写 x 时触发 Store Page Fault，内核走 COW 分配新页。
32
33 child 输出 x=2，parent 最终仍输出 x=1。
34
35 若在 COW handler 里打印了 COW-copy 计数，此处计数 应 +1。

```

cow_multi.c

```

1  #include <ulib.h>
2  #include <stdio.h>
3
4  int main(void) {
5      int x = 3;
6      cprintf("[cow_multi] parent pid=%d initial x=%d (addr=%p)\n", getpid(),
7      x, &x);
8
9      for (int k = 0; k < 3; k++) {
10         int pid = fork();
11         if (pid == 0) {
12             // child k
13             cprintf("[cow_multi-child%d] pid=%d before write x=%d\n", k,
14             getpid(), x);
15             x = 300 + k;
16             cprintf("[cow_multi-child%d] pid=%d after write x=%d\n", k,
17             getpid(), x);
18             exit(0);
19         }
20         waitpid(pid, NULL);
21         cprintf("[cow_multi-parent] after child%d x=%d\n", k, x);
22     }
23
24     cprintf("[cow_multi] parent done pid=%d final x=%d\n", getpid(), x);
25     return 0;
26 }
27
28 /*
29 T4: 多子进程共享同一页: ref>2 的正确复制链
30 child1 写 => 触发复制一次
31
32 child2 写 => 也应触发复制一次 (因为 parent 仍持有旧页 ref>1)
33
34 parent 仍 z=100
35
36 child1 z=101; child2 z=102
37
38 COW-copy 计数应 +2
39 */=

```

cow_parent_write.c

```

1  #include <ulib.h>
2  #include <stdio.h>
3
4  int main(void) {
5      int x = 2;
6      cprintf("[cow_parent_write] start pid=%d x=%d (addr=%p)\n", getpid(), x,
7      &x);
8
9      int pid = fork();

```

```

9     if (pid == 0) {
10         // child: wait a bit so parent writes first
11         for (volatile int i = 0; i < 100000; i++);
12         cprintf("[parent_write-child] pid=%d sees x=%d\n", getpid(), x);
13         exit(0);
14     }
15
16     // parent writes
17     x = 200;
18     cprintf("[parent_write-parent] pid=%d wrote x=%d\n", getpid(), x);
19
20     waitpid(pid, NULL);
21     return 0;
22 }
23 /*
24 T2: 父写也能触发复制（对称性）
25
26 parent 写 y 触发 COW: parent y=20
27
28 child 最终仍看到 y=10
29
30 COW-copy 计数 +1
31 */

```

cow_ro.c

```

1  #include <ulib.h>
2  #include <stdio.h>
3
4  int main(void) {
5      const char *s = "readonly-string";
6      cprintf("[cow_ro] start pid=%d s=%s (addr=%p)\n", getpid(), s, s);
7
8      int pid = fork();
9      if (pid == 0) {
10         cprintf("[cow_ro-child] pid=%d s=%s\n", getpid(), s);
11         exit(0);
12     }
13
14     waitpid(pid, NULL);
15     cprintf("[cow_ro-parent] pid=%d s=%s\n", getpid(), s);
16     return 0;
17 }
18 /*
19 T3: 只读页不应 COW（不会触发 fault）
20 不产生 Store Page Fault
21
22 COW-copy 计数不变
23
24 父子均打印同样字符串
25 */

```

这样之后还会有个问题，我们需要查看这四个文件的输出日志，但由于这四个文件在grade.sh里并没有对应的判断结果。为了解决这个问题，我去询问了大模型，它告诉我需要使用这三行指令来输出日志(以cow_basic.c为例)：


```

21 PMP1: 0x0000000000000000-0xffffffffffffffff (A,R,W,X)
22 DTB Init
23 HartID: 0
24 DTB Address: 0x82200000
25 Physical Memory from DTB:
26   Base: 0x0000000080000000
27   Size: 0x0000000080000000 (128 MB)
28   End: 0x0000000087ffffff
29 DTB init completed
30 (THU.CST) os is loading ...
31
32 Special kernel symbols:
33   entry 0xc020004a (virtual)
34   etext 0xc0205928 (virtual)
35   edata 0xc02cee70 (virtual)
36   end   0xc02d331c (virtual)
37 Kernel executable memory footprint: 845KB
38 memory management: default_pmm_manager
39 physcial memory map:
40   memory: 0x08000000, [0x80000000, 0x87ffffff].
41 vmap_offset is 18446744070488326144
42 check_alloc_page() succeeded!
43 check_pgdir() succeeded!
44 check_boot_pgdir() succeeded!
45 use SLOB allocator
46 kmalloc_init() succeeded!
47 check_vma_struct() succeeded!
48 check_vmm() succeeded.
49 ++ setup timer interrupts
50 kernel_execve: pid = 2, name = "cow_basic".
51 Breakpoint
52 [cow_basic] start pid=2 x=1 (addr=0x801000)
53 [cow_basic-child] pid=3 before write x=1
54 [cow_basic-child] pid=3 after write x=2
55 [cow_basic-parent] pid=2 after child exit x=1
56 all user-mode processes have quit.
57 init check memory pass.
58 kernel panic at kern/process/proc.c:542:
59   initproc exit.

```

其他的日志也有输出了，但我们现在只是重点关注我们想要输出的部分，即从 `++ setup timer interrupts` 这一部分开始，下面我将把剩下三个结果展示出来：

cow_parent_write

```

1  ++ setup timer interrupts
2  kernel_execve: pid = 2, name = "cow_parent_write".
3  Breakpoint
4  [cow_parent_write] start pid=2 x=2 (addr=0x7fffffd8)
5  [parent_write-parent] pid=2 wrote x=200
6  [parent_write-child] pid=3 sees x=200
7  all user-mode processes have quit.
8  init check memory pass.
9  kernel panic at kern/process/proc.c:542:
10   initproc exit.

```

cow_ro

```
1  ++ setup timer interrupts
2  kernel_execve: pid = 2, name = "cow_ro".
3  Breakpoint
4  [cow_ro] start pid=2 s=readonly-string (addr=0x8008a0)
5  [cow_ro-child] pid=3 s=readonly-string
6  [cow_ro-parent] pid=2 s=readonly-string
7  all user-mode processes have quit.
8  init check memory pass.
9  kernel panic at kern/process/proc.c:542:
10     initproc exit.
```

cow_multi

```
1  ++ setup timer interrupts
2  kernel_execve: pid = 2, name = "cow_multi".
3  Breakpoint
4  [cow_multi] parent pid=2 initial x=3 (addr=0x7fffffcc)
5  [cow_multi-child0] pid=3 before write x=3
6  [cow_multi-child0] pid=3 after  write x=300
7  [cow_multi-parent] after child0 x=300
8  [cow_multi-child1] pid=4 before write x=300
9  [cow_multi-child1] pid=4 after  write x=301
10 [cow_multi-parent] after child1 x=301
11 [cow_multi-child2] pid=5 before write x=301
12 [cow_multi-child2] pid=5 after  write x=302
13 [cow_multi-parent] after child2 x=302
14 [cow_multi] parent done pid=2 final x=302
15 all user-mode processes have quit.
16 init check memory pass.
17 kernel panic at kern/process/proc.c:542:
18     initproc exit.
```

说明该用户程序是何时被预先加载到内存中的？与我们常用操作系统的加载有何区别，原因是什么？

“预先”加载

用户程序在 `exec` 阶段（`load_icode`）被提前装入内存，而不是在 `RUNNING` 后再加载，这样在程序执行之前所有需要的内容就已经被加载到内存中了。

与常用操作系统的加载的异同

常用 OS（Linux/windows 等）里：

- `execve` 时从**磁盘文件系统**读取可执行文件；
- 往往是**按需加载/缺页再读**（`demand paging`），并不一定一次性把所有段拷进物理内存；
- 文件页还能用 `page cache` 复用。

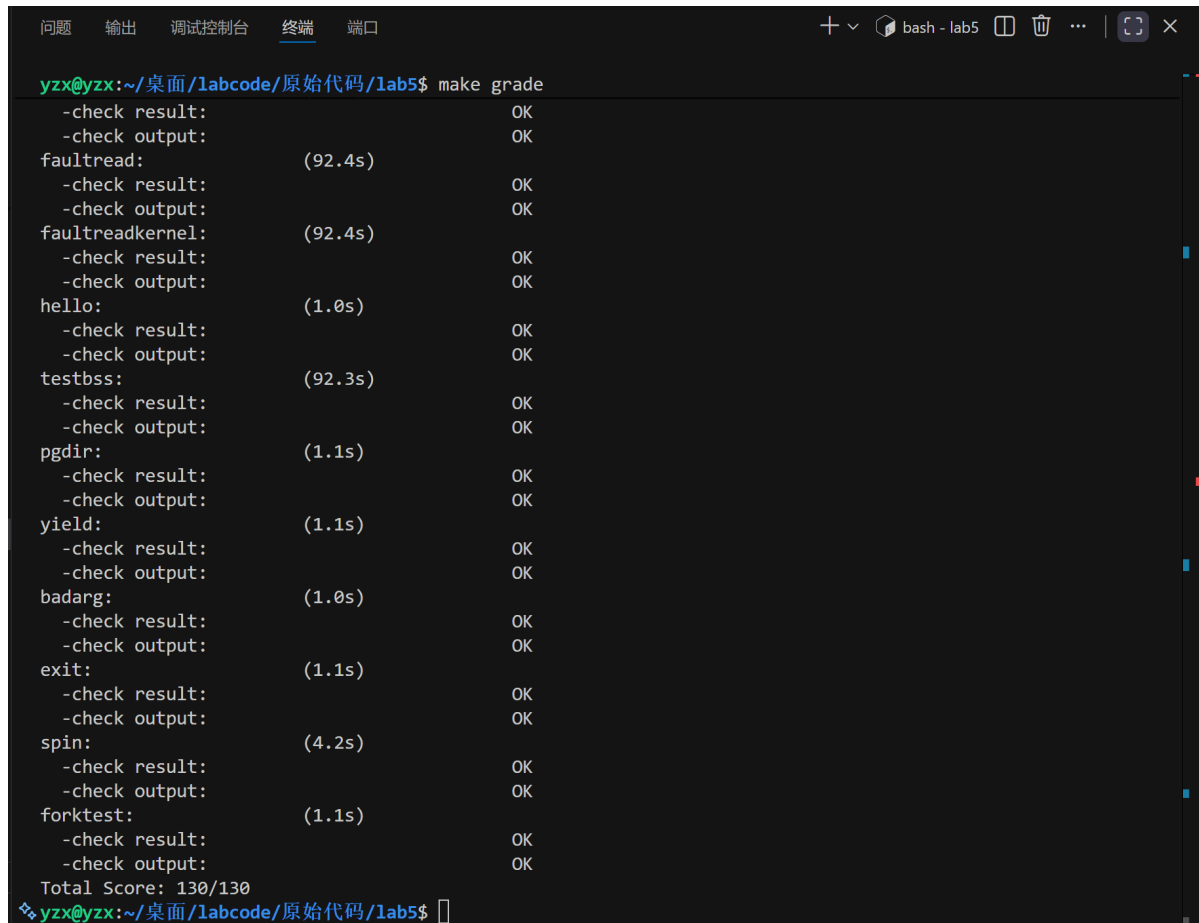
uCore：

- 用户程序通常是编译后直接打包进内核镜像/ramdisk，内核从内存里拿到 ELF，不经过真实磁盘文件系统。
- load_icode() 直接为 ELF 的各个段分配物理页并 memcpy 进去，基本是一次性装完，没有复杂的按需分页/缓存。

原因

- ucore 没有完整的磁盘驱动 + 文件系统支持
- 虚拟内存与按需加载机制尚未完整实现，在实验阶段更容易用“全量拷贝段到内存”来保证可控、可调试。

五、实验结果



```
yzx@yzx:~/桌面/labcode/原始代码/lab5$ make grade
-check result: OK
-check output: OK
faultread: (92.4s)
-check result: OK
-check output: OK
faultreadkernel: (92.4s)
-check result: OK
-check output: OK
hello: (1.0s)
-check result: OK
-check output: OK
testbss: (92.3s)
-check result: OK
-check output: OK
pgdir: (1.1s)
-check result: OK
-check output: OK
yield: (1.1s)
-check result: OK
-check output: OK
badarg: (1.0s)
-check result: OK
-check output: OK
exit: (1.1s)
-check result: OK
-check output: OK
spin: (4.2s)
-check result: OK
-check output: OK
forktest: (1.1s)
-check result: OK
-check output: OK
Total Score: 130/130
yzx@yzx:~/桌面/labcode/原始代码/lab5$
```

六、知识点总结

- 在lab5的实现中，我们是如何维护不同进程之间的关系的？

相较于之前的进程控制块结构体的实现，在lab5中我们新增了cptr、optr、yptr这个属性，分别用于记录该进程的最年轻的子进程(child pointer)、相邻的比它创建更早的的兄弟进程(older sibling pointer)以及相邻的比它的创建更晚的兄弟进程(younger sibling pointer)。为了在进程创建过程中良好地维护各个进程的这三个指针，lab5中调整了set_links函数如下，用以在进程初始化时调用并维护进程控制块中的进程关系指针，必要注释已附：

```
1 static void
2 set_links(struct proc_struct *proc)
3 {
4
5     list_add(&proc_list, &(proc->list_link));
```

```

6      //proc is the youngest in its siblings,so no yptr temporarily
7      proc->yptr = NULL;
8      //its older sibling is the nowa youngest child of its parent
9      if ((proc->optr = proc->parent->cptr) != NULL)
10     {
11         //and wanif it has a optr,optr's yptr shall be set..
12         proc->optr->>yptr = proc;
13     }
14     //the youngest child of parent is proc self
15     proc->parent->cptr = proc;
16     nr_process++;
17 }

```

每当使用do_fork创建进程时，都会调用这个维护函数，从而保证后续在调度过程中，可以快速找到合适的进程以供执行，更重要的是当调用do_exit函数杀死一个

进程时，若其为用户进程，我们可以调用其进程关系指针的信息，将其所有子进程过继给init_proc，从而实现健康有序的进程释放。

- 在lab5中，系统调用是如何实现的？

我们不妨以本次实验主干中的user_main()调用KERNEL_EXECVE(exit)的流程为例来解释系统调用的实现：

该调用经过一系列宏处理后，实际会调用：`kernel_execve(EXIT, _binary_obj__user_exit_start, (size_t)_binary_obj__user_exit_out_size);`函数(函数体及必要注释已经附)。

```

1  // kernel_execve - do SYS_exec syscall to exec a user program called by
   user_main kernel_thread
2  static int
3  kernel_execve(const char *name, unsigned char *binary, size_t size)
4  {
5      int64_t ret = 0, len = strlen(name);
6      // ret = do_execve(name, len, binary, size);
7      asm volatile(
8          "li a0, %1\n"//i",SYS_exec
9          "lw a1, %2\n"//m name
10         "lw a2, %3\n"//m len
11         "lw a3, %4\n"//m binary
12         "lw a4, %5\n"//m size
13         "li a7, 10\n"//sys call interrupt flag,when a7==10,ebreak will run
   special logic
14         "ebreak\n"    //call ebreak,
15
16         //case CAUSE_BREAKPOINT:
17         // cprintf("Breakpoint\n");
18         // if (tf->gpr.a7 == 10)    //since tis a ebreak,a7 will be stored
   into tf.gpr,a7
19         // {
20         //     tf->epc += 4;
21         //     syscall();
22         //         tf &tf + sizeof(struct(tf))
23         //     kernel_execve_ret(tf, current->kstack + KSTACKSIZE);
24         // }
25         // break;
26
27         "sw a0, %0\n"

```

```

28         //0//output
29         : "=m"(ret)
30         //input
31         //1 ==4          %2          %3          %4          %5
32         : "i"(SYS_exec), "m"(name), "m"(len), "m"(binary), "m"(size)
33         //
34         : "memory");
35     //write res of ebreak into ret
36     cprintf("ret = %d\n", ret);
37     return ret;
38 }

```

在该函数中，我们会将系统调用号(此次为SYS_exec)、函数名(name)、二进制文件指针(binary)、函数名长度(len)和二进制代码长度(size)以汇编语言的形式，依序压入a0-a4参数寄存器，同时，我们会依照trap的约定和取巧的硬编码，将a7的值设置为10，以此在触发ebreak时提醒中断处理函数：这里是一个硬编码的系统调用，请帮我做特殊处理，调用syscall()。值得一提，在触发ebreak的过程中，由于前序的工作，之前压入的参数会被STORE_ALL入tf中，以便传递与恢复。将输出定向到ret中。

在syscall中，我们以一个函数入口点数组的形式，保存了每一个系统调用号对应的函数入口点如下：

```

1  static int (*syscalls[])(uint64_t arg[]) = {
2      [SYS_exit]          sys_exit,
3      [SYS_fork]          sys_fork,
4      [SYS_wait]          sys_wait,
5      [SYS_exec]          sys_exec,
6      [SYS_yield]         sys_yield,
7      [SYS_kill]          sys_kill,
8      [SYS_getpid]         sys_getpid,
9      [SYS_putc]           sys_putc,
10     [SYS_pgdir]          sys_pgdir,
11 };

```

然后在负责dispatch的syscall()函数中，通过当前进程tf中的信息确定该选用哪个处理函数，作参数的处理与压入，通过 `tf->gpr.a0 = syscalls[num](arg);`；

调用合适的处理函数。在此次调用中，SYS_exec为系统调用号4的宏，会分发执行sysexec函数，进入具体的二进制函数的加载与执行流程。

这样的实现具有非常好的可扩展性，后续如果需要增加新的系统调用实现，我们只需要在syscall.c中新增处理函数，并为其补足系统调用号即可，

此外，由于传入的参数是以数组形式保存，若新的系统调用需要五个及以上的参数，也只需要在syscall()分发函数中添加参数数组的维护，而不需要改变其他系统调用处理函数的逻辑。

- 在do_execve函数中，我们如何将二进制的函数加载到进程中执行？

在do_execve函数中，我们首先对现在进程的内存结构等关键信息进行检查，对mm进行清理，然后开始执行load_icode复制函数，这个函数是主干的逻辑：

在这里，我们首先为当前进程初始化一个新的mm及页表，拿到可执行elf文件的头elfhdr* elf，并根据其中记录的偏移量拿到程序头列表的入口点proghdr，

并进一步拿到程序数及程序头列表的end，完成以上初始化后，我们开始这样的迭代：

对于程序头列表中每一个元素指向的程序，作映射，检查是否需要加载，设置权限，拿到当前程序的虚拟地址起点p_va及将要写入的页的地址，分配页然后循环进行写入，每写满一页，再分配下一页写入。

以上循环结束后，留下的关于写入的有效信息即为start==end记录了写入的信息的终点，la为最后写入页的下一页的起始地址。

为零初始化在文件中没定义，但是在内存中有需要的BSS部分，我们还需要进行一次迭代，对于处在[p_va + p_filesz, p_va + p_memsz)这之间的内存作零初始化，

这个迭代的具体逻辑和上一个迭代类似，只是这里对目标作零初始化，上一个迭代需要按照elf文件的内容做写入，我们不再赘述。

完成上述工作后，二进制文件内容的加载工作就基本完成了，此后剩下的就是用户栈的内存分配、在其顶上初始化四个可用页，最后将刚才辛苦搓好的mm及页表挂载到当前进程上。

最后设置栈顶，设置用户程序入口点到epc，这样返回后就能直接执行加载到的代码，调整一些中断相关的寄存器标志位使得sret后回到U-Mode，就完成了加载工作(若全部加载成功，会返回0)。

注意，直到load_icode返回、do_execve返回、sys_exec返回、syscall()返回、执行到kernel_execve_ret后，才会回归到__trapret的语义，将刚才存到tf中的入口点等信息恢复出来，然后回归到结束了陷入的过程后，开始执行用户代码。